DDS Nonlinear Chirp Programming guide v 2.0

(Last Modified 02JUN2010)

Part One: Overview

The primary goal of this programming guide is to help understand how to achieve fast nonlinear frequency chirp and amplitude modulation using the homemade DDS, commercial PCI 6534 digital outboards and C++ based codes.

As the general principle, a large number of 32-bit data array (8 Million maximum) is pre-computed and loaded into PCI on board buffer (FIFO, 32MByte); when the PCI digital output card receives a trigger, it flushes out the data at a rate 5MHz max successively (The specified max rate is 20MHz). 26 out of 32 channels of controlled signals are used to address the DDS in real time.

Achieved nonlinear chirp and amplitude modulation rates are (figure 1): set PCI internal clock at 1MHz (experimentally verified with our configurations, >5MHz rates could cause NULL data transfer), Delta Frequency Word (DFW, slope of a linear chirp) can be updated every 6 us and amplitude can be updated every 2 us. A one step change of both the slope and amplitude will require 8 us. For small range frequency chirp, the DFW MSB bits stay unchanged; this rate can be increased by about a factor of 2 or 3. Further improvement of update rate would be implementation of strobe generator, which could lead to 3-fold increase.

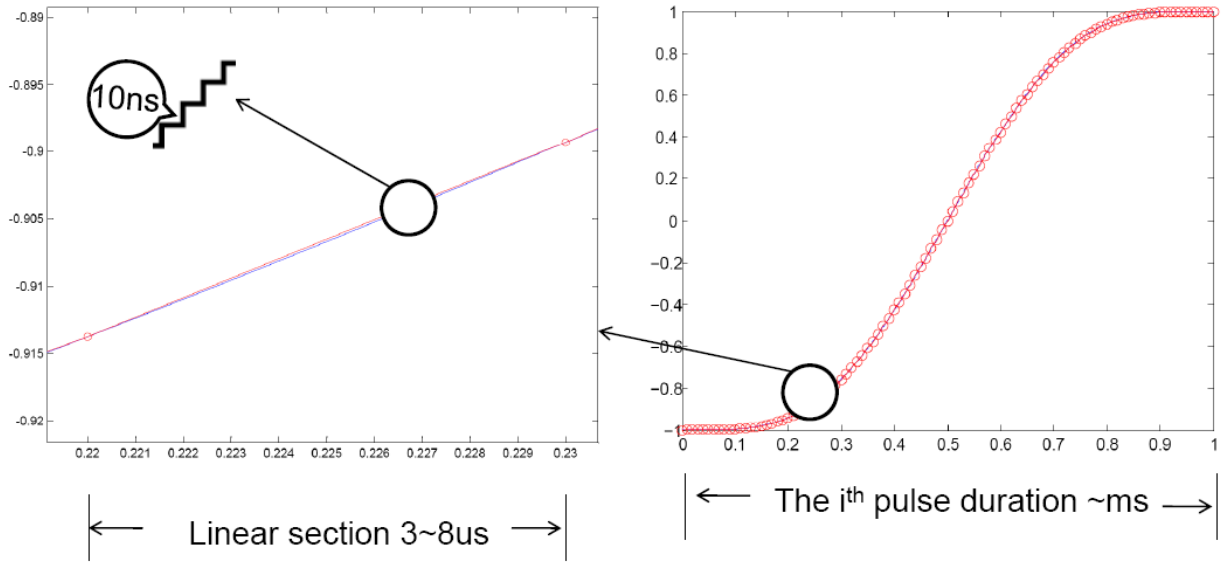


Figure 1 DDS Nonlinear chirp diagram. The numbers are experimentally tested max ratings. The max error in the center of all sections for a typical pulse length of 1ms is about 10~30 ppm (w.r.t. frequency sweeping range).

Part Two: Compute Data Array

The DDS to SCB board connection is made such that it divides computation of data array in a decoupled manner:

Port B 0:5 ➔ AD9852 register address, 6-bit

Port C 0:7 ➔ AD9852 register data, 8-bit

Port D 0:1 ➔ DDS Option number, 2 bit

Port D 2:7 ➔ DDS number, 6-bit

Port A 0:1 ➔Strobe source and self-timing mask

Port A 4:5 ➔ Hold/Shaped Keying

Each data set (unsigned long int, 32-bit) is a sum of 4 Hex strings and finally converted into integer. Unused channels are set to zero. The use of strings enables easy addition and subtraction.

All DDS functions have been written in a C++ header file. Users do not need to know more about these functions except knowing the arguments and returned values (see appendix for details).

Each data set (data i) lasts for 1 clock cycle, while in the middle of the clock cycle the strobe is set to be ON. Data transfer occurs when Strobe is HIGH, which protects DDS from receiving wrong data dissimilar to that of programmed. (*see instructions below on how to generate Strobe*)

Programming flow charts for single tone and nonlinear chirp are shown in figure 2. Based on power up, during master rest all registers are reset to default values. You do NOT need to waste effort to re-write registers that you never need to change.

Special tips: (1)For master reset, 1us duration is enough. (2)For finite pulse length, you need to program power up shape/time, typically 0.5ms from zero to full amplitude. If you force the DDS to start immediately (say 1us) , the power vs time in the next 0.5ms is not guaranteed to be right. (3)During chirp mode, FTW1 is not allowed to change, therefore for multiple pulses with different start frequencies, use one data set DFW in one cycle to switch to next start frequency.

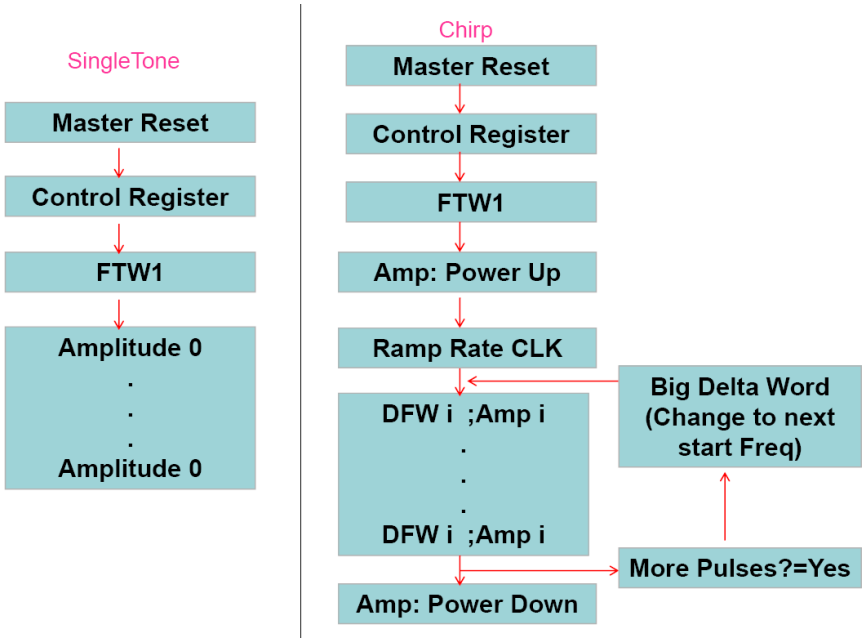


Figure 2 DDS programming flow chart

Part Three: Output Data Array-Recommended Mode of operation

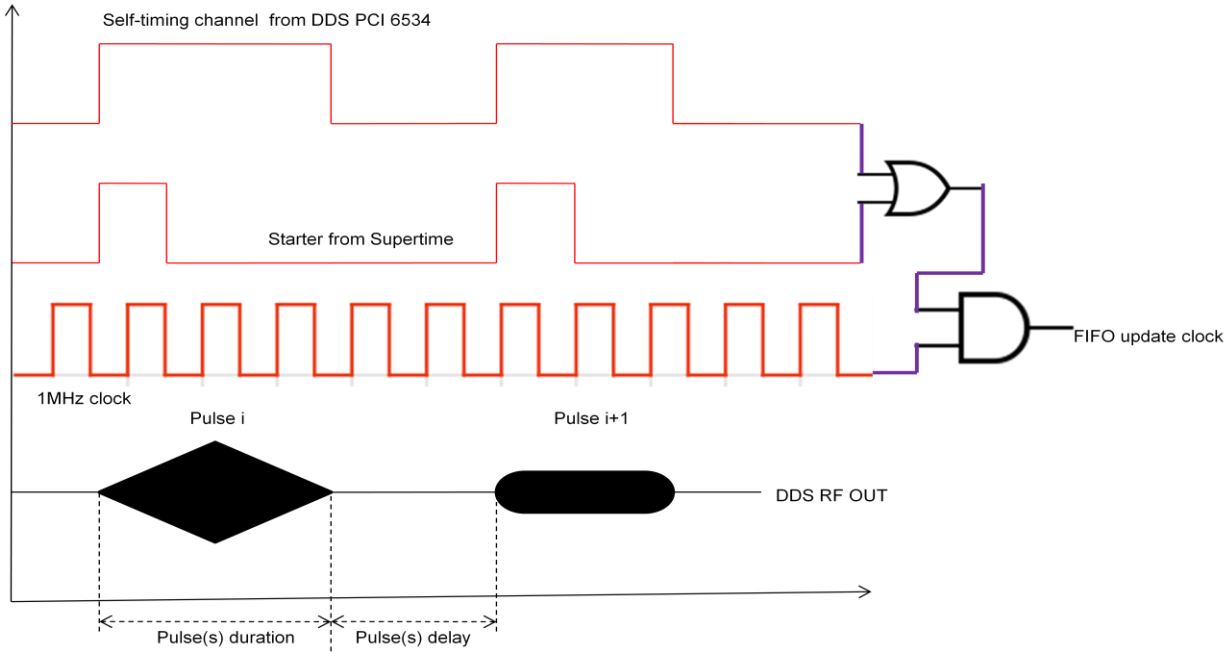


Figure 3 DDS timing control

Under this mode the PCI internal on-board clock is disabled. With a total FIFO size of 32MB and update rate of 1MHz, the total duration of output without delay can reach 8 seconds. For each pulse, the amplitude, frequency and single pulse duration can be made arbitrary according to previous tested parameters. The delay between pulse(s) can be made arbitrary (bigger than 10us).

The code to use external update clock:

```
DAQmxCfgSampClkTiming(taskHandle,"/Dev1/PFI2",Rate,DAQmx_Val_Rising,DAQmx_Val
_FiniteSamps,sample_num); //sample clock on REQ1
```

Once data for all pulses are flushed onto FIFO, the start/stop of a specfic pulse can be precisely timed without any difficulty by using timing control as in Fig.3:

(1) when PCI6534 is in “FIFO” ready state, Supertime sends a start trigger to the OR gate, which enables the FIFO update CLK;

(2)The self-timing mask from one channel of PCI6534 become HIGH when the commonication between PCI6534 and DDS starts; when the comminication stops and the pulse reaches its end, the self-timing mask channel writes LOW to the OR gate and disables the FIFO update CLK.

In this way, supertime does not need to know how long the pulse would be and the start trigger can be as ashort as a few microseconds (but shpuld not be longer than the pulse duration).

How to generate Strobe: a simplest way to strobe the date transfer is shown below, using one channel of PCI6534 as strobe source. At the end of each data computation program, add a line so that: if data index is odd, write HIGH; if index is even write LOW. The strobe generator saves data transfer time and FIFO usage by a factor of 3.

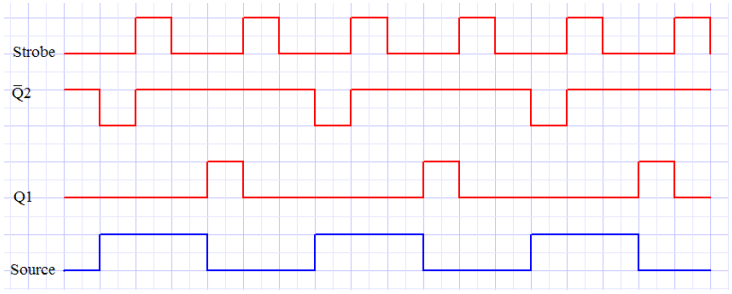


Fig.4 Strobe generator (red wavefroms are output bursts from three multivibrators #74HCT123)

Part Four: C++ to Labview Implementation Options

The current control programs base on both C++ and NI DAQmx, which receive a trigger from supertime and work on a separate PC. It cannot be implemented into NI-DAQ based supertime. Below are the possible options to integrate with supertime:

(1)It is possible to write a simpler Labview program (based on NI DAQ) to load data and trigger the output. The data can be pre-computed with C++ programs.

(2)On the same computer where supertime is installed, downgrade the PCI driver from NI DAQmx to NI DAQ. Then modify the <Output Data Array> part of the C++ program to generate a executable file. Write a link labview program to start this EXE everytime you loop supertime once.

The transition from NI DAQ to NI DAQmx is well exampled here:

http://zone.ni.com/devzone/cda/tut/p/id/5957

(3)One can develop an advanced version of "ddscontrol.vi" (developed by Karl) , which could in general operate at 1MHz update rate and achieve similar chirp rates.

Appendix ("DDSfunc.h")

A. Conversion functions

(A1)**string Dec2Hex(unsigned __int64 num)**

Function to convert unsigned long int to variable length hex, max length 64-Hex

(A2)**string HexN(unsigned __int64 num, unsigned __int8 N)**

Function to convert unsigned long int to fixed length (size-N) hex, with 0s in the front

(A3)**string HexAdd(string PA,string PB,string PC,string PD)**

Function to add three extra strings to form PD+PC+PB+PA

(A4)**int Str2Int(string str)**

Function to convert size-8 Hex string to int (Hex type)

(A5)**string Bin2Hex(string str)**

Function to convert size-8 binary string to size-2 Hex string

string B2H(string str)

Function to convert size-4 binary string to size-1 Hex string

(A6)**string Hex2Bin(string str)**

Function to convert size-8 hex string to size-32 binary string

string H2B(string str)

Function to convert size-1 Hex string to size-4 binary string

(A7)**double sgn(double x)**

Function to output sign of a variable

B. DDS Special functions

(B1)**string UDC(double period, double DDSCLK)**

Function to compute DDS update clock word (32-bit, 8-Hex). 'period' is in sec and 'DDSCLK' is in Hz. Specify 'period' within allowed range: N=period*DDSCLK/2-1, N[1,2^32-1]

C. Frequency functions

(C1)**double Freq(double f0, double HSR, double tao)**

Function to compute frequency vs time. f0-start frequency in Hz, 2*HSR- frequency sweep range in Hz, tao-rescaled pulse time t/tp (range [0,1])

(C2)**string FTW(double f,double DDSCLK)**

Function to compute DDS frequency tunning word (48-bit, 12-Hex). 'DDSCLK' and 'f' are in Hz, typical format of 'f' is 75123456.123456 (uHz precision)

(C3)**string DFW(double df,double DDSCLK)**

Function to compute DDS delta frequency word (48-bit, 12-Hex). 'DDSCLK' and 'df' are in Hz, typical format of 'df' is 00000025.123456 (uHz precision)

(C4)**string RRC(double period, double DDSCLK)**

Function to compute DDS ramp rate clock word (20-bit, 6-Hex). 'period' is in sec and 'DDSCLK' is in Hz. Specify 'period' within allowed range: N=period*DDSCLK-1, N[1,2^20-1]

D. Amplitude functions

(D1)**double SIN2(double Amax, double tao)**

Function to compute amplitude vs time. Amax-max amplitude(range [0,1]), tao-rescaled pulse time t/tp

(D2)**string ATW(double amp)**

Function to compute DDS amplitude multipiler word (12-bit, 4-Hex), range int[0,4095]

Appendix : special tricks

(1)Compile and run the program

Option A (nice and clean, can go without junk project files, small overall file size, sometimes operate faster)

? Run command prompt "cmd.exe"

? cd C:\Documents and Settings\Neal\Desktop\DAQmx-C++ (directory where the programs are stored)

? cl WriteDigPort.cpp /EHsc NIDAQmx.lib (compile the program if changed)

? WriteDigPort.exe (Excute the program)

Option B (a lot of project files, more error messages if something goes wrong, good for diagnostic programmers)

? Open the vc++ project file "VC_WriteDigPort"

? click->Debug->Start Debugging

(2)Stack overflow for large number of samples

For a C++ program with too large variables (bigger than 1MB typical), in our case we define a 32-bit (4 Bytes)

```
unsigned int Data[Nsamples]; //data array computational
uInt32 DataArray[Nsamples]; //data array converted into PCI readable
```

if Nsamples>250000, it will pop a "stack overflow" error during compiling. The simple solution is to add static before any large variables defined (here we have two).

```
static unsigned int Data[Nsamples]; //data array computational
static uInt32 DataArray[Nsamples]; //data array converted into PCI readable
```

Programming v 3.2 Notes (Last Modified 09DEC2011)

Overview This is generic expansion based on previous version 2.0. Two major changes are made: (1) object-orientated C++ is incorporated throughout the programs, which adds ease for the end customer to use and modify the programmable options; (2) Multithreading technique is added to simultaneously or sequentially start multiple tasks (DDS , PULNIX CCD and image analysis for now) under one MATLAB command, saving time by not clicking or typing multiple commands to run a single shot. In the future one can imagine adding more tasks to this architecture, to name a few: Linear photodiode array acquisition, control of magnetic field cancellation, etc.

Program structure

(1)**DDS** A class of one "Pulse.h" is defined to read all possible variations of a microwave pulse, which is called by another class "PulseTrain.h". The latter define, compute and write data necessary to DIO-FIFO, computation sequence can be found in "Composer.h"- a replica for realizing all RF pulses and controls. Finally, "DDS.cpp" or its functional version "DDS_MT.cpp" executes the pulse train with 3 options:
A. Compute FIFO data and write to log file
B. Read from log file and output to FIFO, then DDS
C. Compute FIFO data and output to FIFO, then DDS directly
Since computation can take long time (1 sec or more) if the pulse train is really long (a few hundreds of ms), a faster way to run the DDS is to first do A; and when experiment is ready, run B.

As of today, pulse types include **Blackman PI pulse** and **adiabatic rapid passage**. **Continuous output can also be used for troubleshooting, but keep in mind: Parts inside or near glass cell may be damaged with continuous exposure to 10W microwave, turn off or disconnect the 10W amplifier before starting continuous mode. Special password and procedures have been applied to the program so that even fluent users will HAVE TO take time to get it running, during which 10W AMP is turned off or disconnected.**

(2)**PULNIX CCD** A class "PULNIX.h" is written based JF's earlier version of "acquire_pulnixModified.c", which controls the CCD camera and return/write a 2D array, directly accessible in MATLAB for further image analysis.

(3)**MULTITHREADING** Projects DDS and CCD are combined in "Main.cpp",and can be started simultaneously, the two devices will get ready and waiting for Supertime trigger. In MATLAB terminal, starting these two programs and image analysis can be done by one command.

All programs are in CCD_DDS folder on EDM2@ C:\Documents and Settings\Neal\Desktop\CCD_DDS
To modify pulse parameters: change and save "PulseTrainParameters"
In matlab, to compute DDS data, run "DDSCOMPUTE" ;
to start DDS/CCD/Analysis, run "RUNDDSCCDANA".